Name: Amoroso, Ivan Karl J.
Section: 3.4 BSIT

Course: IT Elective 2
Date: 11-15-26

Activity 4: Student Management CRUD Application

Code:

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | Form1.cs

StudentCrudApp | StudentCrudApp.StudentRepository | GetAll()

```
using System.Data;
using Microsoft.Data.SqlClient;

namespace StudentCrudApp
{
    // Contains ALL database code for the Student table
    2 references
    public class StudentRepository
    {
        4 references
        private const string ConnectionString =
            "Server=(localdb)\\MSSQLLocalDB;Database=SchoolDb;Integrated Security=True;TrustServerCertificate=Tr

        // CREATE: adds a new student.
        1 reference
        public int Insert(string id, string name, string course)
        {
            const string sql =
                "INSERT INTO Students (StudentId, Name, Course) VALUES (@StudentId, @Name, @Course)";

            using (SqlConnection connection = new SqlConnection(ConnectionString))
            using (SqlCommand command = new SqlCommand(sql, connection))
            {
                command.Parameters.Add("@StudentId", SqlDbType.NVarChar, 20).Value = id;
                command.Parameters.Add("@Name", SqlDbType.NVarChar, 100).Value = name;
                command.Parameters.Add("@Course", SqlDbType.NVarChar, 50).Value = course;

                connection.Open();
                return command.ExecuteNonQuery();
            }
        }
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | Form1.cs

StudentCrudApp | StudentCrudApp.StudentRepository | GetAll()

```
        // READ: returns every student as a DataTable.
        1 reference
        public DataTable GetAll()
        {
            const string sql = "SELECT StudentId, Name, Course FROM Students ORDER BY StudentId";

            DataTable table = new DataTable();

            using (SqlConnection connection = new SqlConnection(ConnectionString))
            using (SqlCommand command = new SqlCommand(sql, connection))
            {
                connection.Open();
                using (SqlDataReader reader = command.ExecuteReader())
                {
                    table.Load(reader);
                }
            }

            return table;
        }

        // UPDATE: changes the name and course of ONE student.
        1 reference
        public int Update(string id, string name, string course)
        {
            const string sql =
                "UPDATE Students SET Name = @Name, Course = @Course WHERE StudentId = @StudentId";

            using (SqlConnection connection = new SqlConnection(ConnectionString))
            using (SqlCommand command = new SqlCommand(sql, connection))
            {
                command.Parameters.Add("@Name", SqlDbType.NVarChar, 100).Value = name;
                command.Parameters.Add("@Course", SqlDbType.NVarChar, 50).Value = course;
                command.Parameters.Add("@StudentId", SqlDbType.NVarChar, 20).Value = id;
```

CreateSchoolDb.sql | Program.cs | **StudentRepository.cs** | Form1.cs

StudentCrudApp | StudentCrudApp.StudentRepository | GetAll()

```
62
63                  connection.Open();
64                  return command.ExecuteNonQuery();
65              }
66          }
67
68          // DELETE: removes ONE student.
            1 reference
69          public int Delete(string id)
70          {
71              const string sql = "DELETE FROM Students WHERE StudentId = @StudentId";
72
73              using (SqlConnection connection = new SqlConnection(ConnectionString))
74              using (SqlCommand command = new SqlCommand(sql, connection))
75              {
76                  command.Parameters.Add("@StudentId", SqlDbType.NVarChar, 20).Value = id;
77
78                  connection.Open();
79                  return command.ExecuteNonQuery();
80              }
81          }
82      }
83  }
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | **Form1.cs**

StudentCrudApp | StudentCrudApp.Form1 | courseLabel

```
1   using System;
2   using System.Data;
3   using System.Drawing;
4   using System.Windows.Forms;
5   using System.Data.SqlClient;
6   using Microsoft.Data.SqlClient;
7
8   namespace StudentCrudApp
9   {
        3 references
10      public partial class Form1 : Form
11      {
12          // Controls
            2 references
13          private Label studentIdLabel;
            7 references
14          private TextBox studentIdTextBox;
            2 references
15          private Label nameLabel;
            6 references
16          private TextBox nameTextBox;
            2 references
17          private Label courseLabel;
            8 references
18          private ComboBox courseComboBox;
            3 references
19          private Button saveButton;
            3 references
20          private Button updateButton;
            3 references
21          private Button deleteButton;
            3 references
22          private Button clearButton;
            5 references
23          private DataGridView studentsGrid;
            5 references
24          private Label feedbackLabel;
25
26          // Database connection goes here.
            4 references
27          private readonly StudentRepository repository = new StudentRepository();
28
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | **Form1.cs**

StudentCrudApp | StudentCrudApp.Form1 | courseLabel

```
29          public Form1()
30          {
31              BuildForm();
32              Load += Form1_Load;
33          }
34
35          // Part 1: Form Design
36          private void BuildForm()
37          {
38              Text = "Student Management";
39              Font = new Font("Cascadia Code", 10);
40              ClientSize = new Size(620, 560);
41              FormBorderStyle = FormBorderStyle.FixedDialog;
42              MaximizeBox = false;
43              StartPosition = FormStartPosition.CenterScreen;
44
45              // Input group
46              GroupBox inputGroupBox = new GroupBox
47              {
48                  Name = "inputGroupBox",
49                  Text = "Student Information",
50                  Location = new Point(15, 15),
51                  Size = new Size(590, 170)
52              };
53
54              studentIdLabel = new Label { Name = "studentIdLabel", Text = "Student ID:", Location = new Point(15, 37), AutoSize = true };
55              studentIdTextBox = new TextBox { Name = "studentIdTextBox", Location = new Point(130, 34), Size = new Size(280, 27) };
56
57              nameLabel = new Label { Name = "nameLabel", Text = "Student Name:", Location = new Point(15, 82), AutoSize = true };
58              nameTextBox = new TextBox { Name = "nameTextBox", Location = new Point(130, 79), Size = new Size(280, 27) };
59
60              courseLabel = new Label { Name = "courseLabel", Text = "Course:", Location = new Point(15, 127), AutoSize = true };
61              courseComboBox = new ComboBox
62              {
63                  Name = "courseComboBox",
64                  Location = new Point(130, 124),
65                  Size = new Size(280, 27),
66                  DropDownStyle = ComboBoxStyle.DropDownList
67              };
68              courseComboBox.Items.AddRange(new object[] { "BSIT", "BSCS", "BSIS" });
69
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | **Form1.cs**

StudentCrudApp | StudentCrudApp.Form1 | BuildForm()

```
70              inputGroupBox.Controls.AddRange(new Control[]
71              {
72                  studentIdLabel, studentIdTextBox,
73                  nameLabel, nameTextBox,
74                  courseLabel, courseComboBox
75              });
76
77              // Buttons
78              saveButton = new Button { Name = "saveButton", Text = "Save", Location = new Point(15, 200), Size = new Size(95, 34) };
79              updateButton = new Button { Name = "updateButton", Text = "Update", Location = new Point(120, 200), Size = new Size(95, 34) };
80              deleteButton = new Button { Name = "deleteButton", Text = "Delete", Location = new Point(225, 200), Size = new Size(95, 34) };
81              clearButton = new Button { Name = "clearButton", Text = "Clear", Location = new Point(330, 200), Size = new Size(95, 34) };
82
83              // GridView
84              studentsGrid = new DataGridView
85              {
86                  Name = "studentsGrid",
87                  Location = new Point(15, 250),
88                  Size = new Size(590, 220),
89                  ReadOnly = true,
90                  AllowUserToAddRows = false,
91                  AllowUserToDeleteRows = false,
92                  MultiSelect = false,
93                  SelectionMode = DataGridViewSelectionMode.FullRowSelect,
94                  AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill,
95                  RowHeadersVisible = false
96              };
97
98              feedbackLabel = new Label
99              {
100                 Name = "feedbackLabel",
101                 Text = "",
102                 Location = new Point(15, 485),
103                 Size = new Size(590, 60),
104                 AutoSize = false
105             };
106
107             Controls.AddRange(new Control[]
108             {
109                 inputGroupBox, saveButton, updateButton, deleteButton,
110                 clearButton, studentsGrid, feedbackLabel
111             });
112
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | **Form1.cs**

StudentCrudApp | StudentCrudApp.Form1 | BuildForm()

```
112
113             // Connect event handlers
114             saveButton.Click += saveButton_Click;
115             updateButton.Click += updateButton_Click;
116             deleteButton.Click += deleteButton_Click;
117             clearButton.Click += clearButton_Click;
118             studentsGrid.CellClick += studentsGrid_CellClick;
119         }
120
121         // READ: show all students
            1 reference
122         private void Form1_Load(object sender, EventArgs e)
123         {
124             LoadStudents();
125         }
126
127         // Gets the latest rows from the database
            4 references
128         private void LoadStudents()
129         {
130             try
131             {
132                 DataTable table = repository.GetAll();
133                 studentsGrid.DataSource = table;
134             }
135             catch (SqlException ex)
136             {
137                 MessageBox.Show(ex.Message);
138                 ShowMessage("Could not load students. Check that SchoolDb exists and the connection string is correct.", true);
139             }
140         }
141
142         // When row is clicked, copy the values to the input fields
            1 reference
143         private void studentsGrid_CellClick(object sender, DataGridViewCellEventArgs e)
144         {
145             if (e.RowIndex < 0) return;
146
147             DataGridViewRow row = studentsGrid.Rows[e.RowIndex];
148             studentIdTextBox.Text = row.Cells["StudentId"].Value.ToString();
149             nameTextBox.Text = row.Cells["Name"].Value.ToString();
150             courseComboBox.SelectedItem = row.Cells["Course"].Value.ToString();
151         }
152
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | **Form1.cs**

StudentCrudApp | StudentCrudApp.Form1

```
153         // Create: save new student
154         private void saveButton_Click(object sender, EventArgs e)
155         {
156             if (!TryGetInputs(out string id, out string name, out string course)) return;
157
158             try
159             {
160                 repository.Insert(id, name, course);
161                 LoadStudents();
162                 ClearInputs();
163                 ShowMessage("Student " + name + " was added successfully.", false);
164             }
165             catch (SqlException ex) when (ex.Number == 2627 || ex.Number == 2601)
166             {
167                 // Primary key violation
168                 ShowMessage("A student with ID " + id + " already exists. Use a different ID.", true);
169             }
170             catch (SqlException)
171             {
172                 ShowMessage("The student could not be saved. Please try again.", true);
173             }
174         }
175
176         // Update: update existing student
177         private void updateButton_Click(object sender, EventArgs e)
178         {
179             if (!TryGetInputs(out string id, out string name, out string course)) return;
180
181             try
182             {
183                 int rowsAffected = repository.Update(id, name, course);
184
185                 if (rowsAffected == 0)
186                 {
187                     ShowMessage("No student with ID " + id + " was found, so nothing was updated.", true);
188                     return;
189                 }
190
191                 LoadStudents();
192                 ShowMessage("Student " + id + " was updated successfully.", false);
193             }
194             catch (SqlException)
195             {
196                 ShowMessage("The student could not be updated. Please try again.", true);
197             }
198         }
199
```

CreateSchoolDb.sql | Program.cs | StudentRepository.cs | Form1.cs

StudentCrudApp | StudentCrudApp.Form1

```
200         // Delete: delete existing student
            1 reference
201         private void deleteButton_Click(object sender, EventArgs e)
202         {
203             string id = studentIdTextBox.Text.Trim();
204
205             if (string.IsNullOrWhiteSpace(id))
206             {
207                 ShowMessage("Please select a student from the grid to delete.", true);
208                 return;
209             }
210
211             // Ask for confirmation
212             DialogResult answer = MessageBox.Show("Delete this student?", "Confirm", MessageBoxButtons.YesNo);
213             if (answer != DialogResult.Yes) return;
214
215             try
216             {
217                 int rowsAffected = repository.Delete(id);
218
219                 if (rowsAffected == 0)
220                 {
221                     ShowMessage("No student with ID " + id + " was found, so nothing was deleted.", true);
222                     return;
223                 }
224
225                 LoadStudents();
226                 ClearInputs();
227                 ShowMessage("Student " + id + " was deleted.", false);
228             }
229             catch (SqlException)
230             {
231                 ShowMessage("The student could not be deleted. Please try again.", true);
232             }
233         }
234
235         // Clear: clear input fields
            1 reference
236         private void clearButton_Click(object sender, EventArgs e)
237         {
238             ClearInputs();
239             feedbackLabel.Text = "";
240         }
241
242         // Helper: get inputs and validate
            2 references
243         private bool TryGetInputs(out string id, out string name, out string course)
244         {
245             id = studentIdTextBox.Text.Trim();
246             name = nameTextBox.Text.Trim();
247             course = courseComboBox.Text;
248
```

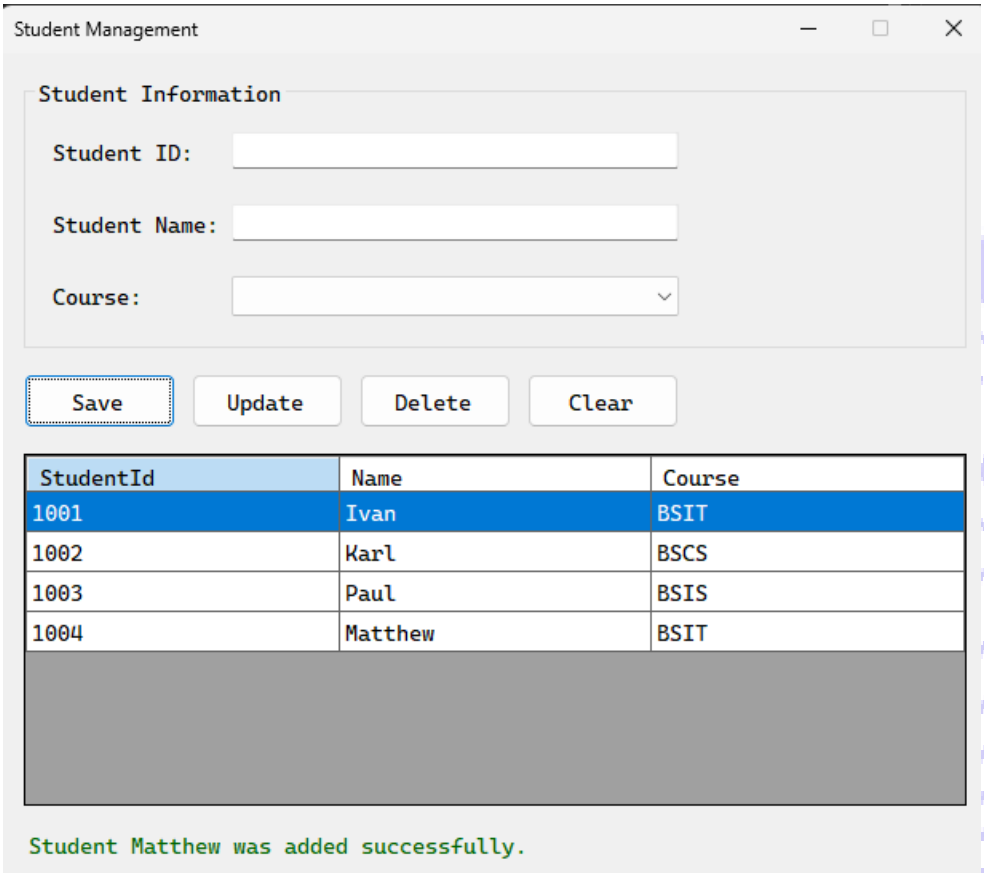
1928

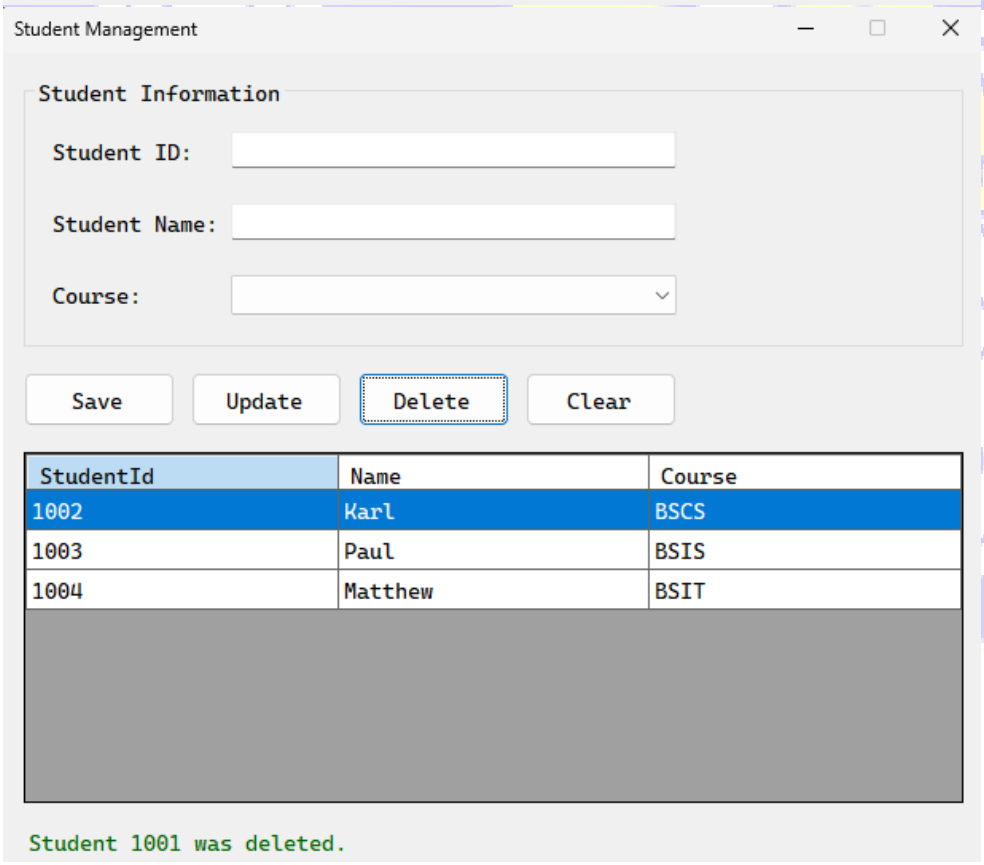
CreateSchoolDb.sql | Program.cs | StudentRepository.cs | Form1.cs

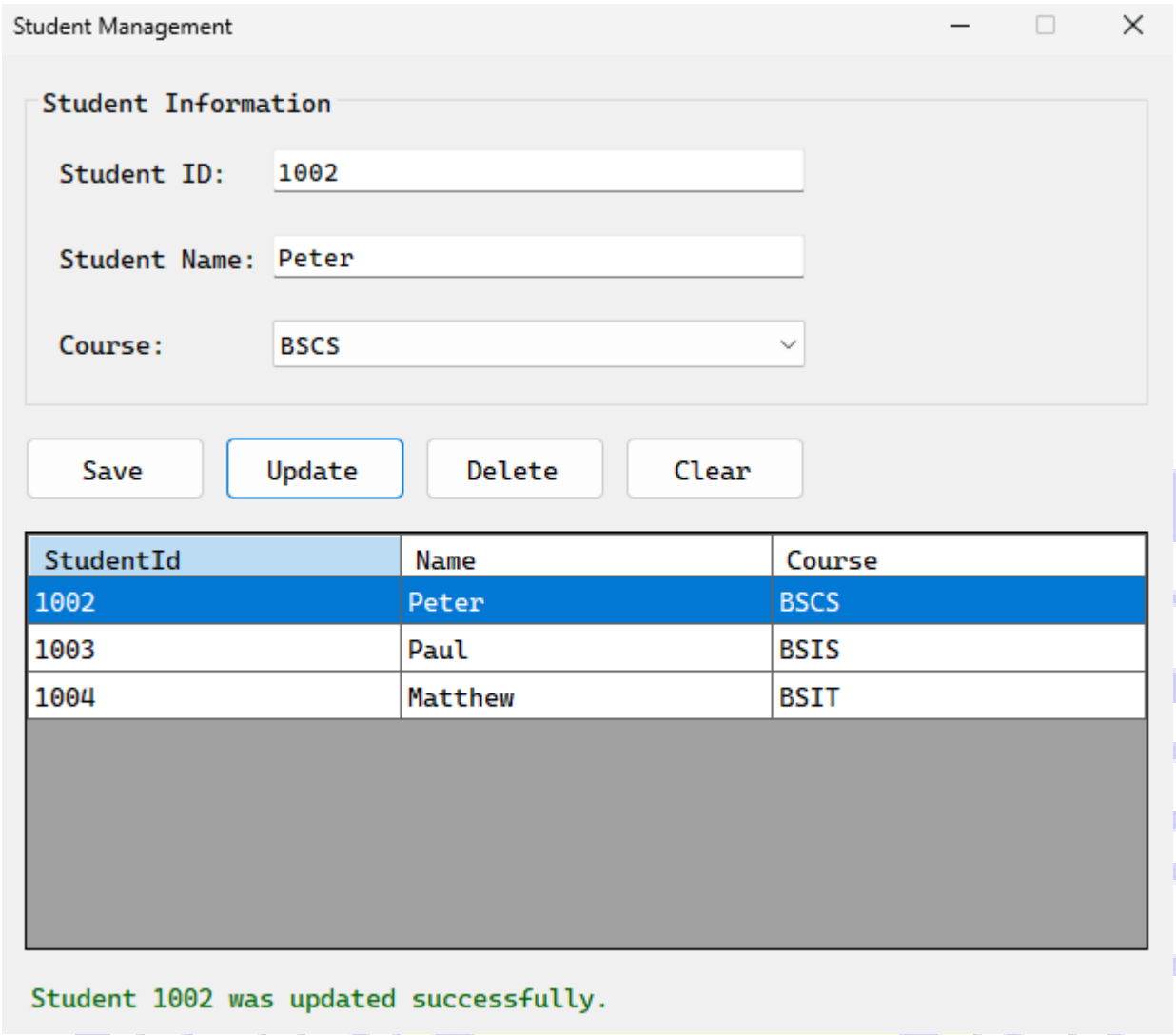
StudentCrudApp | StudentCrudApp.Form1

```
241
242         // Helper: get inputs and validate
            2 references
243         private bool TryGetInputs(out string id, out string name, out string course)
244         {
245             id = studentIdTextBox.Text.Trim();
246             name = nameTextBox.Text.Trim();
247             course = courseComboBox.Text;
248
249             if (string.IsNullOrWhiteSpace(id))
250             {
251                 ShowMessage("Please enter a student ID.", true);
252                 studentIdTextBox.Focus();
253                 return false;
254             }
255
256             if (string.IsNullOrWhiteSpace(name))
257             {
258                 ShowMessage("Please enter a student name.", true);
259                 nameTextBox.Focus();
260                 return false;
261             }
262
263             if (courseComboBox.SelectedIndex == -1)
264             {
265                 ShowMessage("Please select a course.", true);
266                 courseComboBox.Focus();
267                 return false;
268             }
269
270             return true;
271         }
272
273         // Empty the input fields
            3 references
274         private void ClearInputs()
275         {
276             studentIdTextBox.Clear();
277             nameTextBox.Clear();
278             courseComboBox.SelectedIndex = -1;
279         }
280
281         // Shows a message in feedbackLabel
            14 references
282         private void ShowMessage(string message, bool isError)
283         {
284             feedbackLabel.ForeColor = isError ? Color.Firebrick : Color.DarkGreen;
285             feedbackLabel.Text = message;
286         }
287     }
288 }
```

Output:


Student Management
Student Information
Student ID:
Student Name:
Course:
Save
Update
Delete
Clear
StudentId
Name
Course
1001
Ivan
BSIT
1002
Karl
BSCS
1003
Paul
BSIS
1004
Matthew
BSIT
Student Matthew was added successfully.


Student Management
Student Information
Student ID:
Student Name:
Course:
Save
Update
Delete
Clear
StudentId
Name
Course
1002
Karl
BSCS
1003
Paul
BSIS
1004
Matthew
BSIT
Student 1001 was deleted.


Student Management
Student Information
Student ID: 1002
Student Name: Peter
Course: BSCS
Save
Update
Delete
Clear
StudentId
Name
Course
1002
Peter
BSCS
1003
Paul
BSIS
1004
Matthew
BSIT
Student 1002 was updated successfully.